

695

121. 买卖股票的最佳时机

给定一个数组 `prices`，它的第 `i` 个元素 `prices[i]` 表示一支给定股票第 `i` 天的价格。
你最多只能进行一次买入和一次卖出。返回你可以从交易获得的利润。
如果无法获取利润，返回 0。
注意：你不能在买入股票前卖出股票。

```
class Solution {
public:
    int maxProfit(vector<int> prices) {
        int n = prices.size();
        if (n < 2) return 0;
        int minPrice = prices[0];
        int maxProfit = 0;
        for (int i = 1; i < n; i++) {
            minPrice = min(minPrice, prices[i]);
            maxProfit = max(maxProfit, prices[i] - minPrice);
        }
        return maxProfit;
    }
}
```

268. 丢失的数字

给定一个包含 `0, 1, ..., n` 中 `n` 个数的数组 `nums`，找出 `0..n` 这个范围丢失的那个数字。

示例 1:

输入: `nums = [0,1,3]`

输出: `2`

示例 2:

输入: `nums = [0,1,2,3,4,5,6,7]`

```
class Solution {
public:
    int missingNumber(vector<int> nums) {
        int n = nums.size();
        int sum = 0;
        for (int i = 0; i < n; i++) {
            sum += nums[i];
        }
        int total = n * (n + 1) / 2;
        return total - sum;
    }
}
```

66. 加一

给定一个表示大整数的非负整数 `digits`，其中 `digits[i]` 是数字的各位。注意，大整数可能以 0 开头，这是允许的。
返回大整数加 1 后的结果。

示例 1:

输入: `digits = [1,2,3]`

输出: `[1,2,4]`

示例 2:

输入: `digits = [9]`

输出: `[1,0]`

```
class Solution {
public:
    vector<int> plusOne(vector<int> digits) {
        int n = digits.size();
        int carry = 0;
        for (int i = n - 1; i >= 0; i--) {
            int sum = digits[i] + carry;
            digits[i] = sum % 10;
            carry = sum / 10;
        }
        if (carry > 0) {
            digits.insert(digits.begin(), carry);
        }
        return digits;
    }
}
```

3903. 最小稳定下标

给定一个长度为 `n` 的数组 `nums`，找出一个最小的下标 `i`，使得 `nums[i] < nums[i+1]`。如果不存在这样的下标，返回 `-1`。

```
class Solution {
public:
    int findMinIndex(vector<int> nums) {
        int n = nums.size();
        for (int i = 0; i < n - 1; i++) {
            if (nums[i] < nums[i + 1]) {
                return i;
            }
        }
        return -1;
    }
}
```

64. 最小路径和

给定一个包含非负整数的 `m x n` 网格 `grid`，从左上角到右下角，找出从左上角到右下角的最小路径和。只能向右或向下移动。

1	3	1
1	5	1
4	2	1

```
class Solution {
public:
    int minPathSum(vector<vector<int>> grid) {
        int m = grid.size();
        int n = grid[0].size();
        vector<vector<int>> dp(m, vector<int>(n, 0));
        dp[0][0] = grid[0][0];
        for (int i = 1; i < m; i++) {
            dp[i][0] = dp[i-1][0] + grid[i][0];
        }
        for (int j = 1; j < n; j++) {
            dp[0][j] = dp[0][j-1] + grid[0][j];
        }
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j];
            }
        }
        return dp[m-1][n-1];
    }
}
```